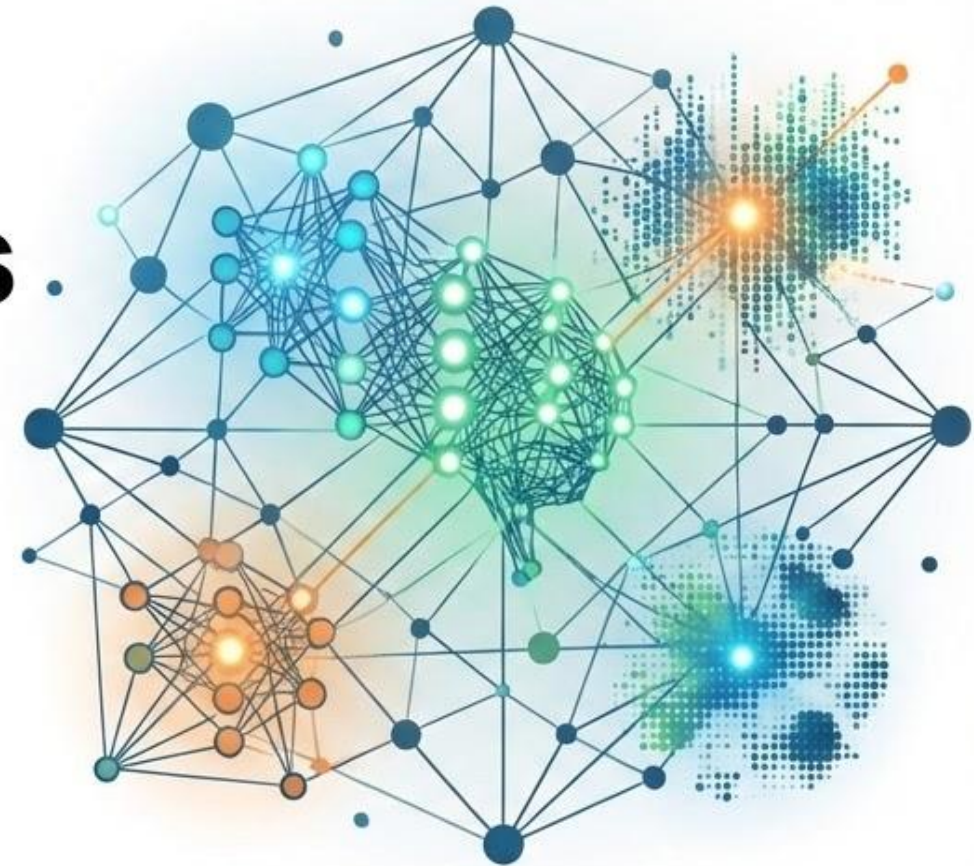


Diplomatura en python para la ciencia de datos y machine learning

MÓDULO 2

Árboles de Decisión y Métricas de Evaluación



Objetivos de la clase

1

Comprender

Qué es un árbol de decisión (nodos, ramas, hojas) y qué es un Bosque Aleatorio, un conjunto de árboles que vota en conjunto.

2

Diferenciar

Los criterios de partición de un árbol (Gini, Entropía, Information Gain) y las dos fuentes de aleatoriedad de un bosque (bootstrap y selección de variables).

3

Controlar

El sobreajuste de un árbol ajustando hiperparámetros y aplicando poda (pruning), y los hiperparámetros clave de un bosque: `n_estimators`, `max_depth` y `max_features`.

4

Evaluar

El desempeño de ambos modelos con accuracy, precision, recall, F1, ROC-AUC y el error Out-of-Bag (OOB).

¿Qué es un árbol de decisión?

Es un modelo predictivo que aprende **reglas simples** tipo "si/entonces" a partir de los datos, dividiendo el espacio de variables en regiones cada vez más puras.

Nodo de decisión: evalúa una condición

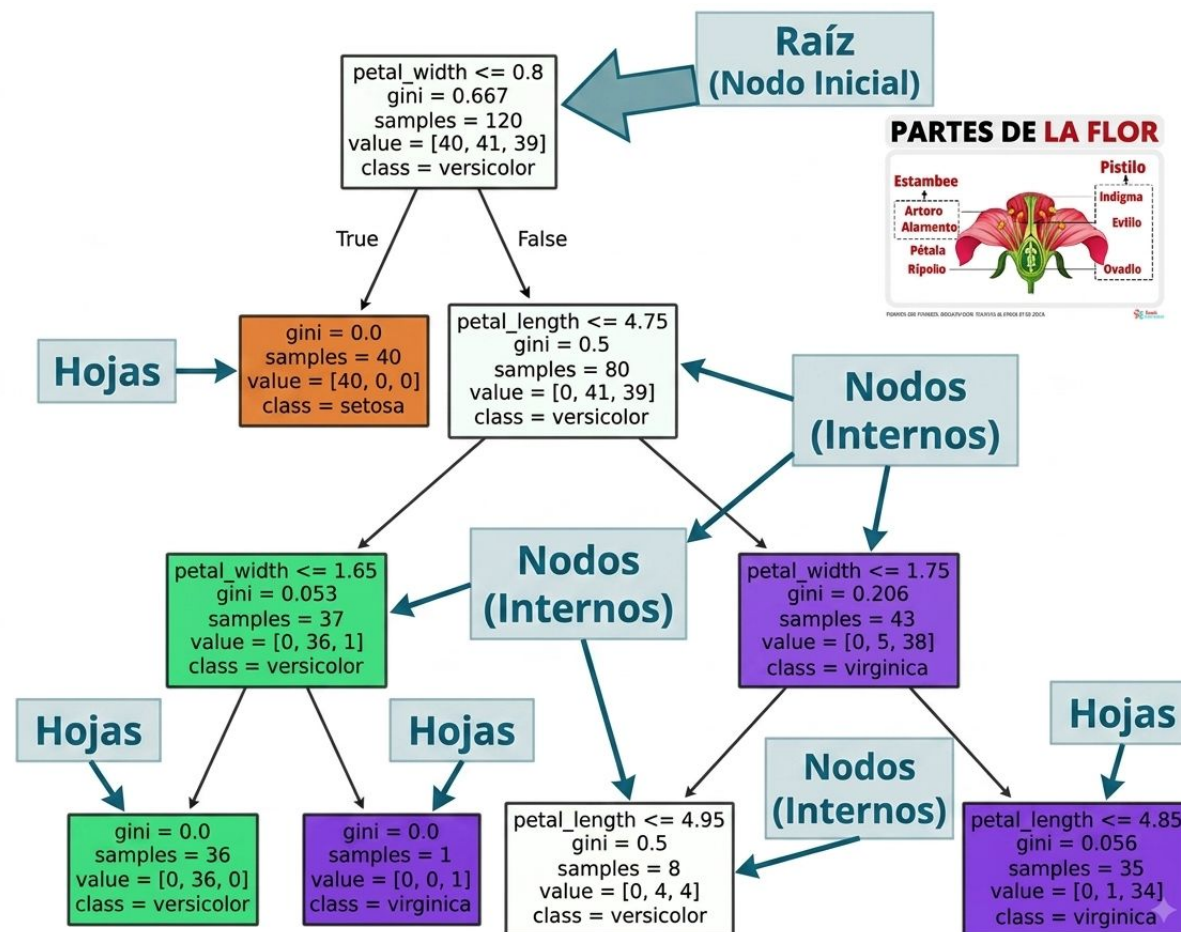
Hoja: asigna una clase final (decisión)

Hoja — asigna una clase final

Ejemplo: Clasificar flores de Iris según su:

- Largo del pétalo
- Ancho del pétalo
- Largo del cáliz
- ancho del cáliz

[Ver el ejemplo](#)



Criterios de partición

Gini Impurity

$$\text{Gini} = 1 - \sum p_i^2$$

Rango: 0 (puro) a 0.5

Mide la probabilidad de clasificar mal una muestra elegida al azar. Es el criterio por defecto en scikit-learn y el más rápido de calcular.

Entropía

$$H = - \sum p_i \cdot \log_2(p_i)$$

Rango: 0 (puro) a 1

Mide el desorden o incertidumbre del nodo, basada en teoría de la información. Su cómputo es algo más costoso que Gini.

Information Gain = **Impureza (padre)** - $\sum (n_i / n) \cdot \text{Impureza}(\text{hijo}_i)$ → se elige el split que maximiza esta ganancia.

Hiperparámetros clave



`max_depth`

Profundidad máxima permitida del árbol.



`min_samples_split`

Muestras mínimas para dividir un nodo interno.



`min_samples_leaf`

Muestras mínimas exigidas en cada hoja.



`criterion`

Función de partición: gini, entropy o log_loss.



`max_features`

Cantidad de variables evaluadas en cada split.



`ccp_alpha`

Controla la poda por complejidad de costo (pruning).

Sobreajuste (Overfitting) y Poda

Un árbol sin restricciones crece hasta memorizar el set de entrenamiento: **accuracy casi perfecto en train, pero bajo en test.**



Pre-pruning

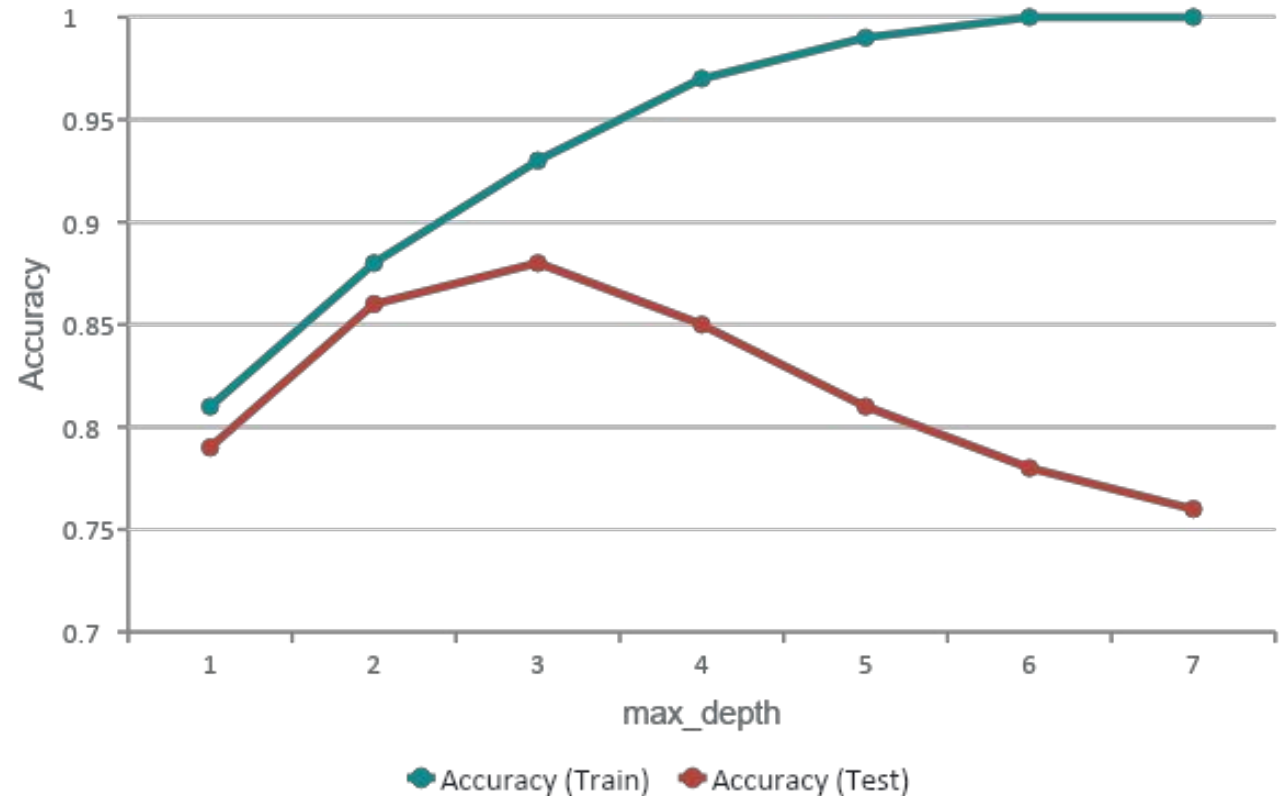
Limitar max_depth, min_samples_leaf, etc. antes de entrenar.



Post-pruning

Entrenar completo y podar después con ccp_alpha (cost-complexity).

Accuracy vs. profundidad del árbol (ejemplo ilustrativo)



Matriz de Confusión

Compara las predicciones del modelo contra el valor real.
Cada celda alimenta el cálculo de accuracy, precision, recall y F1.

- TP** Verdadero Positivo
- FP** Falso Positivo
- FN** Falso Negativo
- TN** Verdadero Negativo



Métricas de Clasificación

Accuracy

$$\frac{(TP + TN)}{Total}$$

% de predicciones correctas en general.

Precision

$$\frac{TP}{(TP + FP)}$$

De lo predicho positivo, cuánto era correcto.

Recall

$$\frac{TP}{(TP + FN)}$$

De lo realmente positivo, cuánto se detectó.

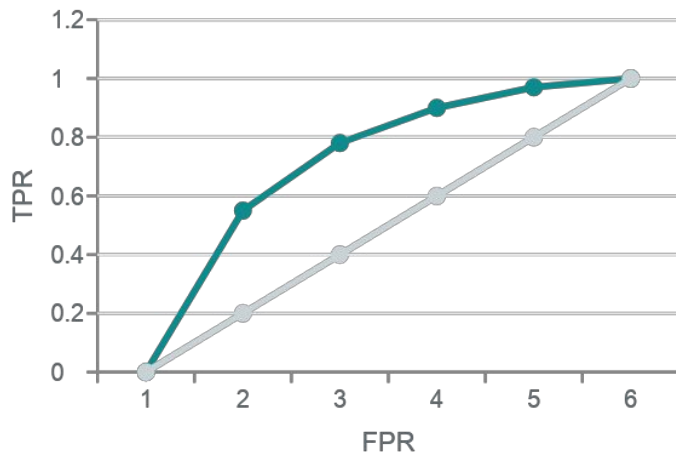
F1-score

$$\frac{2 \cdot (P \cdot R)}{(P + R)}$$

Balance entre precision y recall.

Métricas Avanzadas

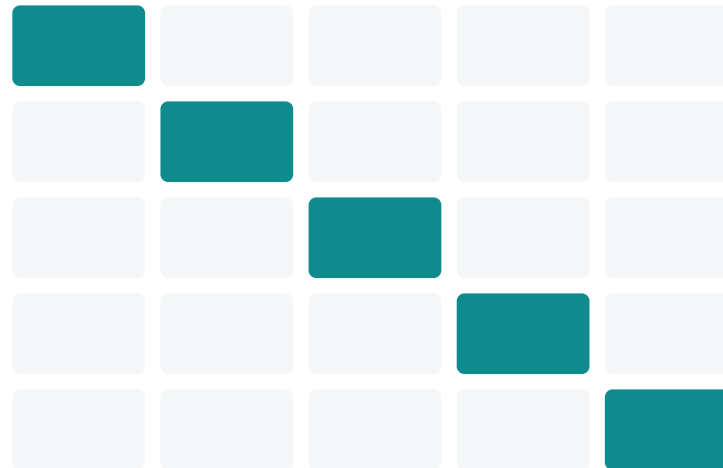
ROC-AUC



AUC = 0.87

Mide la capacidad de separar clases en todos los umbrales posibles. 1.0 = perfecto, 0.5 = azar.

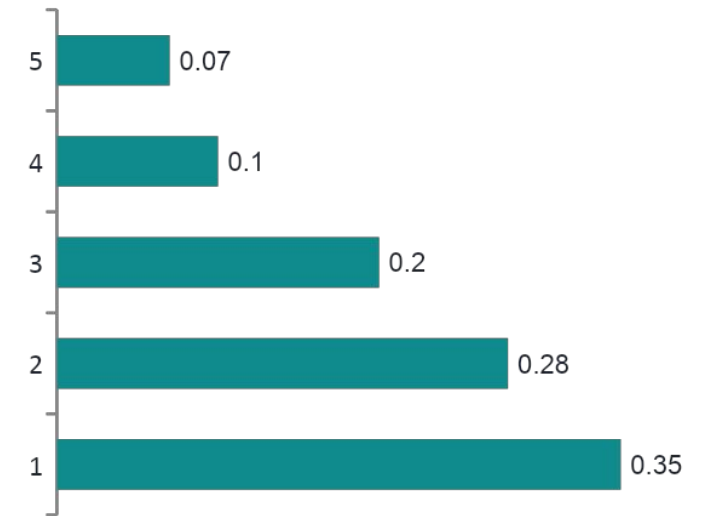
Validación Cruzada (k-fold)



En cada iteración un fold distinto actúa como test (celda oscura) y el resto como entrenamiento.

Reduce la varianza de la estimación del desempeño.

Importancia de Variables



Cuánto reduce cada variable la impureza promedio del árbol (Gini importance).

Ejemplo en Python y Cierre

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

modelo = DecisionTreeClassifier(
    max_depth=5, criterion="gini")
modelo.fit(X_train, y_train)

y_pred = modelo.predict(X_test)
print(classification_report(y_test, y_pred))
```

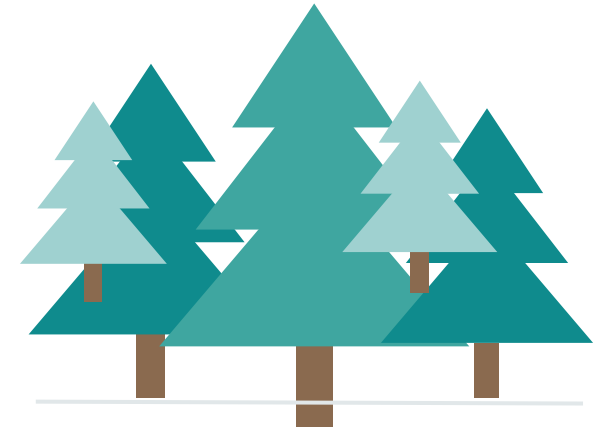
Ideas clave

- ✓ Un árbol divide los datos con reglas simples hasta llegar a hojas puras.
- ✓ Gini / Entropía deciden dónde cortar; el hiperparámetro `criterion` lo controla.
- ✓ Sin límites de profundidad, el árbol sobreajusta: usar poda y validación cruzada.
- ✓ Accuracy no alcanza: revisar precision, recall, F1 y ROC-AUC según el problema.

Ahora que vimos un árbol individual: ¿qué pasa si combinamos muchos? Vamos a Bosques Aleatorios (Random Forest).

Bosques Aleatorios (Random Forest)

Cómo combinar muchos árboles de decisión para lograr predicciones más robustas y menos propensas al sobreajuste.



¿Qué es un Bosque Aleatorio?

Un **Bosque Aleatorio** es un conjunto de muchos Árboles de Decisión, entrenados de forma ligeramente distinta entre sí.

Para predecir, cada árbol "vota" de forma independiente, y el bosque se queda con la **clase más votada** (en clasificación) o con el **promedio** de todas las predicciones (en regresión).

La idea de fondo: "la sabiduría de las masas". Si le preguntás a una sola persona, se puede equivocar; si le preguntás a muchas personas independientes entre sí y te quedás con la respuesta más común, el resultado suele ser más confiable.



[Ver el ejemplo](#)

Las dos fuentes de aleatoriedad

1. Bootstrap (Bagging)

Cada árbol se entrena con una muestra aleatoria del dataset original, elegida CON reemplazo (algunos registros se repiten, otros quedan afuera).

```
bootstrap = True
```

2. Variables aleatorias por corte

En cada partición, el árbol solo puede elegir entre un subconjunto aleatorio de variables (no todas), definido por `max_features`.

```
max_features = "sqrt"
```

Bagging + variables aleatorias = árboles menos parecidos entre sí → sus errores individuales tienden a cancelarse al votar.

El algoritmo, paso a paso



Hiperparámetros clave



n_estimators

Cantidad de árboles que va a tener el bosque.



max_depth

Profundidad máxima de cada árbol individual.



max_features

Cantidad de variables consideradas en cada corte.



min_samples_leaf

Muestras mínimas exigidas en cada hoja.



bootstrap

Si cada árbol se entrena con una muestra con reemplazo (True) o con todo el dataset (False).



criterion

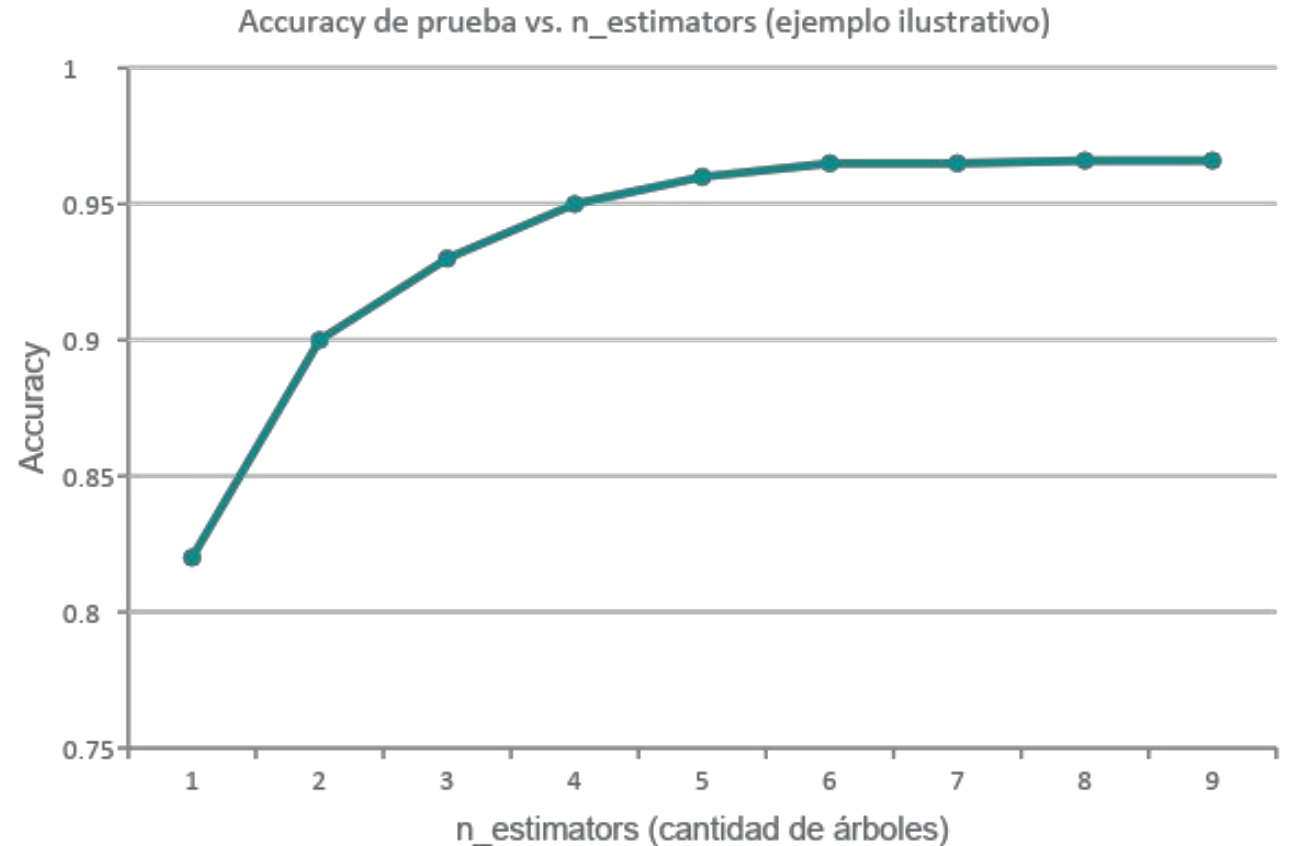
Función de partición de cada árbol: gini, entropy o log_loss.

Menos varianza que un solo árbol

Un árbol de decisión sin límites tiene **alta varianza**: un pequeño cambio en los datos de entrenamiento puede darte un árbol completamente distinto.

Al promediar entre muchos árboles distintos (y algo independientes gracias al bagging), el bosque **cancela buena parte de ese ruido**, sin perder la capacidad de capturar patrones complejos.

A diferencia de `max_depth` en un árbol individual, agregar más árboles (`n_estimators`) casi nunca empeora la generalización del bosque.



Error Out-of-Bag (OOB)

Como cada árbol se entrena con una muestra bootstrap, en promedio deja afuera ("*out-of-bag*") cerca del 37% de los datos originales.

Esos datos "out-of-bag" nunca fueron vistos por ese árbol en particular, así que se pueden usar para evaluarlo -**sin necesitar un conjunto de prueba separado**.

En Scikit-learn:

```
RandomForestClassifier(  
    oob_score=True)
```

Árbol 1



Árbol 2



Árbol 3



Árbol 4



Árbol 5



In-bag (usado para entrenar ese árbol)



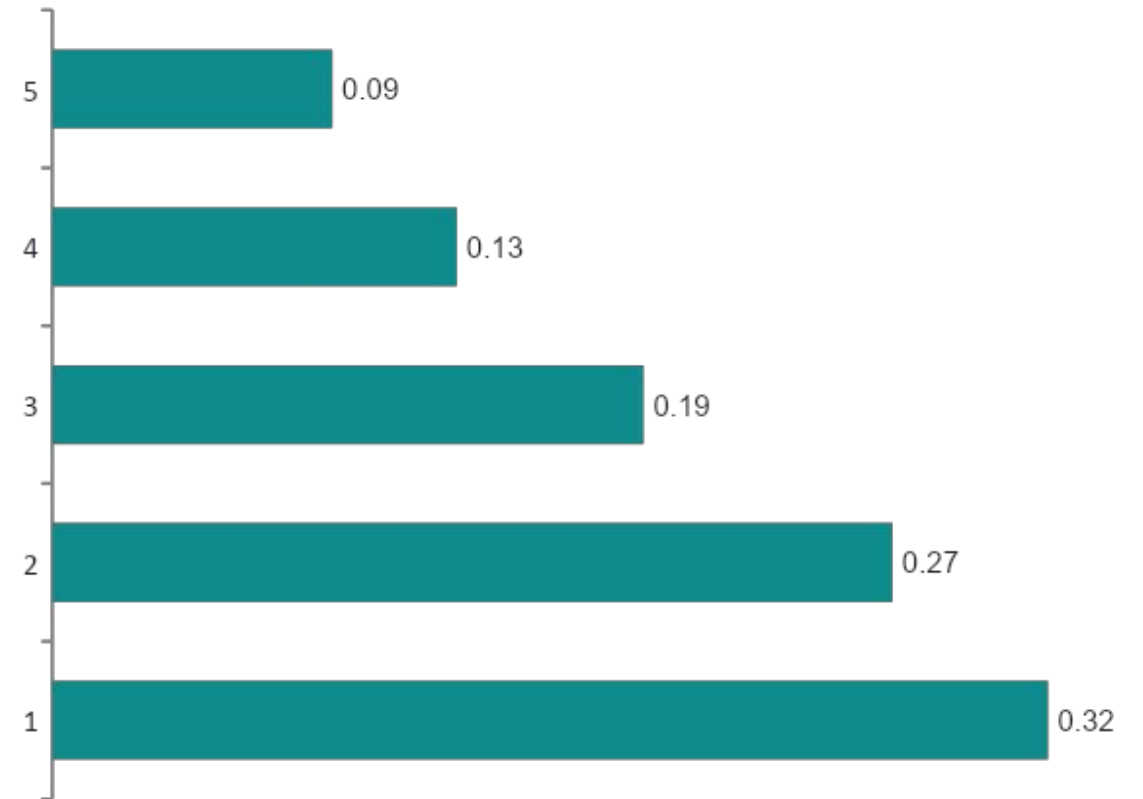
Out-of-bag (sirve para validar)

Importancia de Variables

Cada árbol del bosque calcula su propia importancia de variables (igual que vimos con un árbol individual). Random Forest simplemente **promedia esos valores** entre todos los árboles.

Ventaja frente a un solo árbol: al promediar entre muchos árboles entrenados con datos y variables distintas, el resultado es mucho más estable -no depende tanto de una única muestra de entrenamiento en particular.

Importancia promedio (ejemplo ilustrativo)



Ventajas y Desventajas

Ventajas

- ✓ Muy buen desempeño "de fábrica", con poco ajuste de hiperparámetros.
- ✓ Mucho más robusto al sobreajuste que un árbol individual.
- ✓ Da una medida de importancia de variables, más estable que la de un solo árbol.

Desventajas

- ✗ Mucho menos interpretable que un único árbol de decisión.
- ✗ Más lento para entrenar y predecir (hay que recorrer todos los árboles).
- ✗ Ocupa más memoria: hay que guardar cada uno de los árboles del bosque.

Familia de "primos" → Extra Trees (aún más aleatoriedad) y Gradient Boosting / XGBoost (los árboles se entrenan uno tras otro, corrigiendo los errores del anterior).

Ejemplo en Python y Cierre

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

bosque = RandomForestClassifier(
    n_estimators=200,
    max_depth=None,
    max_features="sqrt",
    oob_score=True,
    random_state=42)
bosque.fit(X_train, y_train)
print(f"OOB score: {bosque.oob_score_:.2f}")

y_pred = bosque.predict(X_test)
print(classification_report(y_test, y_pred))
```

Ideas clave

- ✓ Un Bosque Aleatorio es un conjunto de árboles de decisión que votan juntos.
- ✓ El bagging y la selección aleatoria de variables hacen que los árboles sean distintos entre sí.
- ✓ Más árboles (n_estimators) casi nunca empeora el resultado, solo lo hace más lento.
- ✓ El error Out-of-Bag permite validar el modelo sin separar un conjunto de prueba aparte.

Con esto cerramos KNN, Árboles de Decisión y Bosques Aleatorios

Modelos de Aprendizaje Supervisado

Regresión Lineal	Regresión Logística	k-NN	Árbol de Decisión	Random Forest
REGRESIÓN LINEAL	REGRESIÓN LOGÍSTICA	k-NN	ÁRBOL DE DECISIÓN	RANDOM FOREST
Tipo de Problema Regresión Fortaleza Principal Altamente interpretable y extremadamente rápida de entrenar. Debilidad Crítica Asume relaciones estrictamente lineales en los datos.	Tipo de Problema Clasificación binaria Fortaleza Principal Entrega probabilidades de predicción e interpretabilidad. Debilidad Crítica Solo es capaz de definir fronteras de decisión lineales.	Tipo de Problema Clasificación y Regresión Fortaleza Principal Modelo intuitivo, simple y sin fase de entrenamiento. Debilidad Crítica Predicción lenta en datasets grandes; sensible a la escala de variables.	Tipo de Problema Clasificación y Regresión Fortaleza Principal Muy interpretable visualmente; modela relaciones no lineales. Debilidad Crítica Alta tendencia al sobreajuste si no se limita su crecimiento.	Tipo de Problema Clasificación y Regresión Fortaleza Principal Altamente robusto, evita sobreajuste y mide importancia de variables. Debilidad Crítica Requiere más cómputo; es más lento de entrenar y menos interpretable.